Actor-Critic Implementation Specification

Detailed Design of the Tabular Actor-Critic and Shared-Network Deep A2C
used in the v6 (OLLA-Relative) Case-1 RL-AMC Stack

Anonymous Authors
Department of Electrical and Electronic Engineering

Scope. This document specifies the **v6 (OLLA-relative)** Actor-Critic internals directly. Every learner carries the same per-user OLLA outer loop as the comparator, and the action is an offset from the OLLA-corrected reference (size 5 for standard range ± 2, size 7 for expanded range ± 3), so *action* $0 \equiv$ *OLLA*. The state adds an *OLLA-offset* feature (5 tabular bins / continuous for the deep net) and a scheduler-regime one-hot, giving an 8-D deep input. The optimiser mechanics (log-softmax, n-step returns, advantage standardisation, global-norm clip 5.0, entropy anneal $0.05 \to 0.001$, Adam rates) are identical to the audited v5 machinery. The authoritative parameter set is Table 4 of `Case1_v6_Methodology_and_Architectural_Justification`.

Abstract

This document is the low-level implementation specification for the two Actor-Critic algorithms used in the v6 RL-AMC stack: (a) a shared-parameter tabular Actor-Critic (*AC_Shared_v6*) and (b) a shared-network Deep Advantage Actor-Critic (*DeepA2C_Shared_v6*). Both share the same MDP formulation (OLLA-relative CQI offset with a per-learner OLLA loop, a 6-feature tabular / 8-D deep state, 5-action set) but differ substantially in data structures, initialisation, update rules, and numerical stability tricks. This document covers memory footprints, per-TTI compute, weight initialisation, gradient-flow details, GPU handling, and the eight non-obvious implementation tricks that separate a working implementation from a training-time divergence.

Keywords — Actor-Critic, Advantage A2C, tabular methods, neural policy gradient, MATLAB `dlnetwork`, GPU implementation, CUDA forward compatibility, parameter sharing, gradient clipping.

1 Scope and Preliminaries

The v6 RL-AMC stack implements three algorithms sharing an identical MDP formulation. This document specifies the two Actor-Critic algorithms; the Q-Learning variant is specified separately. Every design decision is either derived from the update equations in Sections 2 and 3, or is a numerical-stability trick documented in Section 4.

The following MDP quantities are shared between both algorithms and are computed identically at every TTI:

- **OLLA offset** $\Delta_{\text{OLLA}}(t)$: a per-user scalar (dB) updated once per scheduled TTI on realised ACK/NACK of the sent CQI ($+\Delta_u$ on ACK, $-\Delta_d$ on NACK, $\Delta_d = 0.5$ dB, $\Delta_u/\Delta_d = B_t/(1-B_t)$, $B_t = 0.1$). Identical to the OLLA comparator's loop.
- **OLLA-corrected reference CQI** $\text{ref}(t) = \text{baseCQI}(\text{obs_SINR}(t) + \Delta_{\text{OLLA}}(t))$, where `baseCQI_v5.m` applies the 3GPP threshold table. *Action 0 transmits at* $\text{ref}(t)$*, i.e. exactly the OLLA decision.*
- **Δ reference** $\Delta\text{ref}(t) = \text{clip}(\text{ref}(t) - \text{ref}(t-1), -5, +5)$.
- **SINR bin** $b_S(t) = \arg\min_i |B_i - \text{obs_SINR}(t)|$, where B is 20 uniform bins over $[-10, 30]$ dB.
- **Trend bin** $b_T(t) \in \{-1, 0, +1\}$: sign-of-difference of $\text{obs_SINR}(t) - \text{obs_SINR}(t-1)$ against edges $\{-1, +1\}$.
- **OLLA-offset bin** $b_\Delta(t) \in \{1, \ldots, 5\}$: $\Delta_{\text{OLLA}}(t)$ discretised at edges $\{-3, -1, +1, +3\}$ dB *(new in v6).*
- **Scheduler regime** $g \in \{\text{PF}, \text{RR}, \text{EQ}\}$: one-hot, observable.
- **Action** $a \in \{-2, -1, 0, +1, +2\}$ (index $\in \{1, \ldots, 5\}$; swept to $\{-3, -2, -1, 0, 1, 2, 3\}$ for expanded range sweeps).

Variable	Shape	Size (float64)
θ (actor preferences)	$15\times11\times20\times3\times5\times3\times5$	742 500 cells (5.9 MB)
V (critic values)	$15\times11\times20\times3\times5\times3$	148 500 cells (1.2 MB)
Total footprint		**≈7.1 MB**

- **Sent CQI** $c_s(t) = \mathrm{clip}(\mathrm{ref}(t) + a, 1, 15)$.
- **Reward** (cooperative, cell-level; CTDE) every user receives the identical $r(t) = \big(\sum_u (1 - \mathrm{BLER}_u(t)) \cdot \mathrm{TBS}_u(t) \cdot 10^{-3}\big)/\rho$, summed over scheduled users; $\rho = 30$ Mbps.

Throughout, N denotes the number of users; both algorithms share their parameters across all N users. The MDP tuple is $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with $|\mathcal{A}| = 5$ (or 7 for expanded actions) and $\gamma = 0.95$.

2 Tabular Actor-Critic (`AC_Shared_v6`)

2.1 Data Structures

The tabular Actor-Critic maintains two multi-dimensional arrays:

Both arrays are shared across all N users — there is no user-index dimension. Every user's transition updates the same θ and V.

2.2 Initialisation

Both tables are initialised to zero. No bias initialisation is applied; the OLLA-relative action encoding already guarantees that an untrained state transmits at the OLLA-corrected reference (action $0 \equiv$ OLLA), so no additional prior is required.

2.3 Policy

The policy is a numerically-stable softmax over $\theta(s, \cdot)$:

$$p_a(s) = \theta(s, a) - \max_{a'} \theta(s, a') \tag{1}$$

$$\pi(a|s) = \frac{\exp(p_a(s))}{\sum_{a'} \exp(p_{a'}(s))} \tag{2}$$

Equation (1) is essential: without the $\max_{a'}$ subtraction, exp overflows to `Inf` once any single $\theta(s, a)$ exceeds ~ 700 (double-precision limit).

Actions are sampled by inverse-CDF sampling on $\pi(\cdot|s)$:

```
u = rand;
csum = cumsum(pi_s);
a_idx = find(u <= csum, 1, 'first');
if isempty(a_idx), a_idx = num_actions; end
```

The fallback to `num_actions` handles the rare floating-point corner case where `sum(pi_s)` < 1 due to underflow.

2.4 Update Rule

Given a transition (s, a, r, s') and TD error $\delta = r + \gamma V(s') - V(s)$:

$$V(s) \leftarrow V(s) + \alpha_v \, \delta \tag{3}$$

$$\theta(s, a) \leftarrow \theta(s, a) + \alpha_\theta \, \delta \, (1 - \pi(a|s)) \tag{4}$$

$$\theta(s, a') \leftarrow \theta(s, a') - \alpha_\theta \, \delta \, \pi(a'|s), \ \forall a' \neq a \tag{5}$$

Equations (4)–(5) implement the vanilla policy gradient $\nabla_\theta \log \pi(a|s) = e_a - \pi(\cdot|s)$ where e_a is the standard basis vector for the taken action. In code this is written compactly as one vector update:

Symbol	Value	Purpose
α_θ	0.05	Actor learning rate
α_v	0.10	Critic learning rate
γ	0.95	Discount factor
ρ	30 Mbps	Reward normalisation
Batch size	1 (online TD(0))	
N_{episode}	400	Training episodes
Simulation length	200 TTI/episode	
Seeds	3 (paper), ≥ 5 recommended	

```
grad_log = -pi_s; % - pi(.|s)
grad_log(a_idx) = grad_log(a_idx) + 1; % + e_a
theta(s_ref, s_dref, s_sinr, s_trend, s_olla, s_sched, :) = ...
    squeeze(theta(s_ref, s_dref, s_sinr, s_trend, s_olla, s_sched, :)) + ...
    alpha_theta * delta * grad_log;
```

The `squeeze`/`:` indexing is a MATLAB-specific way of promoting a 5-element vector into a length-5 slice of the 7-D array (six state dimensions + the action axis).

2.5 Hyperparameters

2.6 Full Pseudocode

```
Algorithm 1 -- Tabular Actor-Critic (Shared across N users)
------------------------------------------------------------
Input:  simTime T, num_episodes E, env, seed
Output: theta, V, learning_curve

Initialise theta = 0, V = 0
Initialise learning_curve[1..E] = 0

for episode = 1 to E:
    sched = sample from env.train_schedulers (PF-weighted) // Domain rand.
    sc    = schedContext(sched)                            // one-hot index
    (S_true, dist)  = generateChannel_v5(T, env)
    (obs_SINR, ~)   = cqiFeedback_v5(S_true, env)
    olla(1..N) = 0                                         // per-user OLLA offset
    prev_ref = baseCQI(obs_SINR(1, :) + olla)
    R_emas = zeros(1, N)                                   // EMA for PF
    ep_tp  = 0

    for t = 1 to T:
        ref(1..N)  = baseCQI(obs_SINR(t, :) + olla)        // OLLA-corrected
        dref(1..N) = clip(ref - prev_ref, -5, +5)

        // --- ACTOR: sample action for each user ---
        for i = 1 to N:
            (sb, tb) = binIndices(obs_SINR(t, i), trend(t, i))
            ob       = ollaBin(olla(i))                    // 5-bin (v6)
            prefs = theta(ref(i), dref(i)+6, sb, tb, ob, sc, :)
            prefs = prefs - max(prefs)                     // (eq. 1)
            pi_i  = exp(prefs) / sum(exp(prefs))           // (eq. 2)
            a_i   = sampleInverseCDF(pi_i)
            store (i, ref(i), dref(i)+6, sb, tb, ob, sc, pi_i, a_i)

        // --- ENVIRONMENT: compute BLER, TBS, potential rate ---
        for i = 1 to N:
            sent(i) = clip(ref(i) + action_set(a_i), 1, 15) // action 0 == OLLA
            bler(i) = estimateBLER_v5(S_true(t, i), sent(i)-1)
            pot(i)  = (1-bler(i)) * TBS(sent(i)-1, RB_full)

        // --- SCHEDULER: decide who transmits ---
        rb = schedule_users_v5(sched, t, N, R_emas, pot, 110)

        // --- ACHIEVED TP, OLLA offset update, cell reward ---
        cell_goodput = 0
        for i = 1 to N:
            if rb(i) > 0:
                tp_i = (1-bler(i)) * TBS(sent(i)-1, rb(i))
```

Layer	Actor	Critic
Input	8 features	8 features
Hidden 1 (FC + ReLU)	$8 \to 64$	$8 \to 64$
Hidden 2 (FC + ReLU)	$64 \to 64$	$64 \to 64$
Output layer	$64 \to 5$ (logits)	$64 \to 1$ (linear)
Softmax	applied at loss-time	–
Trainable weights	$8 \cdot 64 + 64^2 + 64 \cdot 5 = 4928$	$8 \cdot 64 + 64^2 + 64 = 4672$
Trainable biases	$64 + 64 + 5 = 133$	$64 + 64 + 1 = 129$
Total params	$\approx 5\,061$	$\approx 4\,801$

```
            cell_goodput += tp_i
            if rand > bler(i): olla(i) += d_up          // ACK
            else:              olla(i) -= d_down         // NACK
        else: tp_i = 0
        R_emas(i) = (1-emaA)*R_emas(i) + emaA*tp_i
    r_cell = cell_goodput / rho                         // CTDE: same r for all
    ep_tp += cell_goodput

    // --- TD UPDATE per user (shared theta, V) ---
    ref_next  = baseCQI(obs_SINR(t+1, :) + olla)        // t=T -> t
    dref_next = clip(ref_next - ref, -5, +5)
    for i = 1 to N:
        (sb', tb') = binIndices(obs_SINR(t+1, i), trend(t+1, i))
        ob' = ollaBin(olla(i))
        V_s   = V(ref(i), dref(i)+6, sb, tb, ob, sc)
        V_sp  = V(ref_next(i), dref_next(i)+6, sb', tb', ob', sc)
        delta = r_cell + gamma * V_sp - V_s
        V(ref(i), dref(i)+6, sb, tb, ob, sc) += alpha_v * delta
        grad_log = -pi_i
        grad_log(a_i) += 1
        theta(ref(i), dref(i)+6, sb, tb, ob, sc, :) += ...
            alpha_theta * delta * grad_log

    prev_ref = ref

  learning_curve(episode) = ep_tp / T

return theta, V, learning_curve
```

2.7 Compute Complexity

Per TTI, per user: $O(1)$ policy evaluation (one softmax over 5 actions), one BLER lookup, one TBS lookup, one TD update. Total per TTI: $O(N)$. Per episode: $O(NT)$. Per training run: $O(NTE)$.

For $N = 50$, $T = 200$, $E = 400$: 4 million TD updates per training run. Wall-clock ≈ 30 seconds per seed on a single CPU core.

3 Deep Advantage Actor-Critic (`DeepA2C_Shared_v6`)

The deep variant replaces the tabular (θ, V) with two neural networks. All N users share both networks. Training uses n-step Advantage Actor-Critic [1] with several stability tricks described in Section 4.

3.1 Network Architectures

Both actor and critic have identical hidden layers; they differ only in output.

The output layer of the actor produces raw *logits*. Softmax is applied inside the loss function using the numerically-stable log-softmax formulation (Section 4, Trick 3), not inside a softmax layer. This is deliberate.

3.2 State Vector

$$\begin{aligned} s(t) = \big[\, &\mathrm{ref}(t)/15,\ \Delta\mathrm{ref}(t)/5,\ \mathrm{obs_SINR}(t)/30,\ dS(t)/5, \\ &\Delta_{\mathrm{OLLA}}(t)/5,\ \mathrm{onehot}(g)\,\big]^\top \\ dS(t) = \mathrm{obs_SINR}(t) &- \mathrm{obs_SINR}(t-1) \end{aligned} \tag{6}$$

The 8-D input comprises four continuous features (OLLA-corrected reference CQI, its first difference, observed SINR, and the continuous SINR difference dS), the continuous OLLA offset Δ_{OLLA} *(new in v6)*, and a 3-way scheduler-regime one-hot. Deep A2C uses the *continuous* dS and Δ_{OLLA} rather than the 3-bin trend and 5-bin offset used by the tabular AC; both encode the same quantities, and the network exploits the finer resolution.

3.3 Weight Initialisation and Precision

MATLAB's `dlnetwork` default initialisation is used (Glorot for linear layers, zero for biases). All parameters are then converted to single-precision `gpuArray`s if a compatible GPU is available:

```
if gpuDeviceCount > 0
    try
        parallel.gpu.enableCUDAForwardCompatibility(true);
    catch
    end
    gpuDevice;
    actorNet = dlupdate(@(x) gpuArray(single(x)), actorNet);
    criticNet = dlupdate(@(x) gpuArray(single(x)), criticNet);
end
```

The `enableCUDAForwardCompatibility(true)` call is required for CUDA compute capability sm_120 (Blackwell / RTX 50-series). Without it, the GPU is detected but `predict` fails.

3.4 Loss Functions

For a batch of $B = n_{\text{step}} \cdot N$ transitions $\{(s_i, a_i, r_i, s'_i)\}$, define:

- *Bootstrap value*: $V_{\text{boot}} = V_w(s_T)$ if the trajectory did not terminate; else 0.
- *n-step returns* (per user, walked backwards): $R_k = r_k + \gamma R_{k+1}$, $R_n = V_{\text{boot}}$.
- *Advantage*: $A_k = R_k - V_w(s_k)$.
- *Standardised advantage* (Trick 4, Section 4): $\tilde{A}_k = (A_k - \bar{A})/(\sigma_A + 10^{-6})$.
- *Log-softmax* (Trick 3): $\log \pi_\theta(a|s) = \text{logit}(s,a) - \log \sum_{a'} \exp \text{logit}(s,a')$.

Losses:

$$\mathcal{L}_{\text{critic}}(w) = \frac{1}{B} \sum_k A_k^2 \tag{7}$$

$$\mathcal{L}_{\text{actor}}(\theta) = -\frac{1}{B} \sum_k \log \pi_\theta(a_k|s_k) \cdot \text{stopgrad}(\tilde{A}_k) - \beta_t H(\pi_\theta) \tag{8}$$

$$H(\pi_\theta) = -\sum_a \pi_\theta(a|s) \log \pi_\theta(a|s) \tag{9}$$

The `stopgrad` on $\tilde{A}$ is critical: the advantage is a critic target, not something the actor should backpropagate through. In MATLAB this is implemented as `dlarray(extractdata(advantage))`.

3.5 Hyperparameters

3.6 Full Pseudocode

```
Algorithm 2 -- Shared-Network Deep A2C (Shared over N users)
------------------------------------------------------------
Input:  simTime T, num_episodes E, env, seed
Output: actorNet, criticNet, learning_curve, entropy_curve

Build actorNet: 4->64->relu->64->relu->5 (logits)
Build criticNet: 4->64->relu->64->relu->1 (linear)
If GPU available:
    parallel.gpu.enableCUDAForwardCompatibility(true)
    Move both nets to gpuArray(single)

```

Symbol	Value	Notes
γ	0.95	Discount
n_{step}	10	Rollout length
lr_actor	5×10^{-4}	Adam
lr_critic	1×10^{-3}	Adam
Adam β_1	0.9	Default
Adam β_2	0.999	Default
Entropy β_t	$0.05 \rightarrow 0.001$ (linear anneal)	Over episodes
Gradient clip	$\|g\|_2 \leq 5.0$ (global)	Both nets
Reward norm. ρ	30 Mbps	Same as Tab AC
N_{episode}	400	
Simulation length	200 TTI/episode	
Seeds	3 (paper), ≥ 5 recommended	

```
Initialise Adam state for actor and critic (empty avg + avg-sq)
Initialise learning_curve[1..E] = 0, entropy_curve[1..E] = 0

Allocate rollout buffer:
    buf_states  = zeros(8, n_step * N, 'single')    // 8-D state (v6)
    buf_actions = zeros(1, n_step * N)
    buf_rewards = zeros(1, n_step * N, 'single')
    buf_count   = 0

for episode = 1 to E:
    entW = linearAnneal(0.05, 0.001, episode, E)     // Entropy schedule
    sched = sample from env.train_schedulers (PF-weighted)
    ctx   = onehot(schedContext(sched))              // 3-way, repeated over N
    (S_true, ~) = generateChannel_v5(T, env)
    (obs_SINR, ~) = cqiFeedback_v5(S_true, env)
    olla(1..N) = 0                                   // per-user OLLA offset
    prev_ref = baseCQI(obs_SINR(1, :) + olla)
    ep_tp = 0, ep_ent = 0

    for t = 1 to T:
        ref   = baseCQI(obs_SINR(t, :) + olla)       // OLLA-corrected
        dref  = clip(ref - prev_ref, -5, +5)
        dS    = obs_SINR(t, :) - obs_SINR(t-1, :)      // 0 if t=1
        S     = [ref/15; dref/5; obs_SINR(t,:)/30; dS/5; olla/5; ctx]  // 8 x N

        // --- FORWARD PASS (batched across N users) ---
        S_gpu   = dlarray(gpuArray(single(S)), 'CB')
        logits  = extractdata(predict(actorNet, S_gpu))  // 5 x N
        logits  = double(logits - max(logits, [], 1))
        P       = exp(logits) ./ sum(exp(logits), 1)
        ep_ent += mean(- sum(P .* log(P + 1e-12), 1))

        // --- SAMPLE ACTION per user ---
        for i = 1 to N:
            a_i = sampleInverseCDF(P(:, i))
            sent(i) = clip(ref(i) + action_set(a_i), 1, 15) // action 0 == OLLA
            bler(i) = estimateBLER_v5(S_true(t, i), sent(i)-1)
            pot(i)  = (1-bler(i)) * TBS(sent(i)-1, RB_full)

        // --- SCHEDULER ---
        rb = schedule_users_v5(sched, t, N, R_emas, pot, 110)

        // --- ACHIEVED TP, OLLA update, cell reward, buffer push ---
        cell_goodput = 0
        for i = 1 to N:
            if rb(i) > 0:
                tp_i = (1-bler(i)) * TBS(sent(i)-1, rb(i))
                cell_goodput += tp_i
                if rand > bler(i): olla(i) += d_up      // ACK
                else:              olla(i) -= d_down     // NACK
            else: tp_i = 0
            R_emas(i) = (1-emaA)*R_emas(i) + emaA*tp_i
        r_cell = cell_goodput / rho                      // CTDE: same r for all
        ep_tp += cell_goodput
        for i = 1 to N:
            buf_count += 1
            buf_states(:, buf_count) = S(:, i)
```

```
            buf_actions(buf_count)   = a_i(i)
            buf_rewards(buf_count)   = r_cell

        prev_ref = ref

        // --- A2C UPDATE every n_step TTI or at t=T ---
        if buf_count >= n_step*N or t == T:
            // Time-limit-aware bootstrap (v6): use V(s_T), never a hard 0
            tn = min(t+1, T)
            (ref_next, dref_next, dS_next) = computeNext(obs_SINR, olla, tn)
            sN = [ref_next/15; dref_next/5; obs_SINR(tn,:)/30; dS_next/5; olla/5; ctx]
            V_boot = extractdata(predict(criticNet, dlarray(gpuArray(sN),'CB')))

            // Compute n-step returns per user (backwards)
            K = buf_count
            Tsteps = K / N
            returns = zeros(1, K)
            for i = 1 to N:
                R = V_boot(i)
                for tt = Tsteps downto 1:
                    idx = (tt-1)*N + i
                    R = buf_rewards(idx) + gamma * R
                    returns(idx) = R

            // Gradient step (see next algorithm block)
            [aGrad, cGrad] = dlfeval(@a2c_grads, actorNet, criticNet, ...
                                     buf_states(:,1:K), buf_actions(1:K), ...
                                     returns, entW)
            aGrad = clipGlobalNorm(aGrad, 5.0)
            cGrad = clipGlobalNorm(cGrad, 5.0)
            [actorNet, aAvg, aAvgSq]  = adamupdate(actorNet, aGrad, ..., lr_actor)
            [criticNet, cAvg, cAvgSq] = adamupdate(criticNet, cGrad, ..., lr_critic)
            buf_count = 0

    learning_curve(episode) = ep_tp / T
    entropy_curve(episode)  = ep_ent / T

return actorNet, criticNet, learning_curve, entropy_curve
```

```
Sub-routine -- a2c_grads (inside dlfeval)
------------------------------------------------------------
Input:  actorNet, criticNet, S (batch), A (actions), R (returns), entW
Output: actorGrad, criticGrad

V = forward(criticNet, S)                    // 1 x B
advantage = R - V                             // 1 x B
criticLoss = mean(advantage.^2)

logits = forward(actorNet, S)                 // 5 x B
maxL   = max(logits, [], 1)
logZ   = maxL + log(sum(exp(logits - maxL), 1))
logp_all = logits - logZ                      // log-softmax, 5 x B

// One-hot mask for taken actions
mask = zeros(5, B)
for b = 1 to B: mask(A(b), b) = 1

logp = sum(logp_all .* mask, 1)               // 1 x B, log prob of A(b)

// Standardise advantage (stop-gradient)
adv_d = double(extractdata(advantage))
sd    = std(adv_d)
if sd > 1e-6:
    adv_d = (adv_d - mean(adv_d)) / sd
else:
    adv_d = adv_d - mean(adv_d)
adv_d = dlarray(cast(adv_d, 'like', logits))  // detached

entropy = -sum(exp(logp_all) .* logp_all, 1)  // 1 x B
actorLoss = -mean(logp .* adv_d) - entW*mean(entropy)

criticGrad = dlgradient(criticLoss, criticNet.Learnables, 'RetainData', true)
actorGrad  = dlgradient(actorLoss,  actorNet.Learnables)
return actorGrad, criticGrad
```

3.7 Compute Complexity

Per TTI, per user:

- Forward pass through actor + critic: $O(D)$ where $D \approx 10\,000$ is the parameter count. On a modern GPU this is ~ 1 ms.
- Sample action + environment step: $O(1)$.
- Buffer push: $O(1)$ (amortised).

Every n_{step} TTI, per user:

- Backward pass + Adam update: $O(D)$, ~ 3 ms on GPU.

For the headline $N = 10$, $T = 200$, $E = 500$ (and up to the preliminary $N = 50$): on the order of 10^6 forward passes and 10^5 Adam updates. Wall-clock $\approx$ 10–15 minutes per seed on RTX 5060 Ti.

4 Eight Non-Obvious Implementation Tricks

Each of these was empirically necessary. Removing any one produces either training instability or a wrong result.

Trick 1 — Softmax with max-subtraction

Direct `exp(theta)` overflows once any $\theta(s, a) > 700$. All softmax computations must first subtract $\max_{a'} \theta(s, a')$ (Equation 1). Applies to both tabular AC and Deep A2C.

Trick 2 — Inverse-CDF sampling with fallback

The naive `find(rand <= cumsum(pi))` can return an empty array if $\sum \pi < 1$ due to underflow. Always fall back to the last action index.

Trick 3 — Log-softmax at loss time, not softmax layer

If the actor network ends in a `softmaxLayer` and the loss uses `log(prob)`, the composition `log(exp(logit)/Z)` loses precision whenever $\pi(a|s)$ is small (near zero). Instead, output raw logits and compute

$$\log \pi(a|s) = \text{logit}(s, a) - \text{logsumexp}_{a'} \text{logit}(s, a') \tag{10}$$

directly. This is numerically stable for arbitrarily peaked policies.

Trick 4 — Advantage standardisation with stop-gradient

Per-batch z-score of the advantage: $\tilde{A} = (A - \bar{A}) / \max(\sigma_A, 10^{-6})$. Empirically halves gradient variance and dramatically stabilises training. The advantage must be detached (`stopgrad`) before use in the actor loss — otherwise gradient flows through the critic path and interferes with critic learning.

Trick 5 — Global gradient-norm clipping ($\|g\|_2 \leq 5$)

Compute the L2 norm of the concatenated actor (or critic) gradients across all layers, then rescale if the norm exceeds 5.0. In MATLAB:

```
total = 0;
for i = 1:height(grads)
    g = extractdata(grads.Value{i});
    total = total + sum(g(:).^2);
end
nrm = sqrt(total);
if nrm > 5.0
    for i = 1:height(grads)
        grads.Value{i} = grads.Value{i} * (5.0/nrm);
    end
end
```

Per-tensor clipping is not equivalent and can bias updates toward small-parameter layers. Global is what standard A2C references specify.

Table 1: Symptoms → likely causes → fixes.

Symptom	Likely cause	Fix
Policy collapses to one action	Entropy weight too low; missing gradient clip	Anneal entropy from 0.05; add global clip
Return oscillation, episode Tp swings by $> 50\%$	No advantage standardisation	Add per-batch z-score of A before actor loss
NaN in first few episodes	`softmax(exp(logits))` overflow; missing max-subtract	Add Trick 1; log-softmax at loss time (Trick 3)
Tabular AC learns but underperforms baseline	Zero-offset action $\neq$ OLLA (encoding bug)	Use OLLA-relative encoding: action 0 must transmit at baseCQI(obs_SINR + Δ_{OLLA}) (see main paper)
Deep A2C RNG behaviour depends on GPU	Adam state on GPU while some ops on CPU	Use consistent `gpuArray(single(...))` throughout
Sudden performance drop mid-training	Gradient explosion event; catch shows the loss $\to \infty$	Add global gradient-norm clip (Trick 5)
Different seeds produce very different final Tp	Insufficient seeds; report as noise	Increase n_{seeds} to ≥ 5
Deep A2C wins on cell Tp but bad BLER	Missing entropy or too little clipping	Increase β start value; verify clip norm
Tab AC beats Deep A2C on cell Tp	State too small/discrete for the net to add value, or too few episodes	Verify the OLLA-offset feature is wired in; train longer; under v6 the OLLA-relative state is where A2C leads on PF

Trick 6 — Entropy annealing $\beta_t = 0.05 \to 0.001$

Fixed entropy weight either (a) prevents policy collapse but caps final performance ($\beta = 0.05$) or (b) allows late-stage exploitation but collapses early ($\beta = 0.001$). Linearly annealing between them gives the best of both.

Trick 7 — Time-limit-aware bootstrapping

The episode horizon $t = T$ is an *artificial* time limit, not a true terminal state, so the return is bootstrapped with the critic's estimate $V_{\text{boot}} = V_w(s_T)$ at the end of every n-step segment — *including* at $t = T$ — rather than with a hard 0. Bootstrapping 0 at the artificial horizon biases the returns downward near episode ends (Pardo et al. 2018).

Trick 8 — CUDA forward-compatibility for Blackwell

MATLAB R2024a and earlier reject the RTX 5060 Ti (sm_120) unless `parallel.gpu.enableCUDAForwardCompatibility(t` is called before the first `gpuDevice` touch. Wrap in a `try/catch` to remain compatible with pre-R2023b, which lacks this API.

5 Common Pitfalls (Diagnosis Guide)

6 Verification and Testing

An implementation is considered correct when:

1. **OLLA-relative sanity**: If the agent's actions are forced to $a = 0$ at every TTI, throughput equals the OLLA scheme exactly (bit-for-bit, since both run the same code path). This tests
2. **Zero-shadow test**: With zero rewards, both θ and V remain at zero, and the policy remains uniform. Tests the update equations for accidental bias terms.
3. **Gradient magnitude check**: In the first 5 episodes of Deep A2C, actor and critic gradient norms should be in $[10^{-3}, 10^{+1}]$. Norms outside this range signal initialisation bugs or missing clipping.
4. **Entropy trajectory**: Deep A2C entropy should start near $\log 5 \approx 1.61$ (uniform) and decrease monotonically to $\in [0.3, 0.7]$ over training. Non-monotone decay usually indicates a broken entropy bonus.

5. **Cross-scheduler robustness**: PF-trained policy should retain $\geq 45\%$ of PF throughput under RR evaluation. Lower values indicate insufficient scheduler domain randomisation.

6. **Reproducibility**: Two runs with identical seeds should produce cell-Tp values within ± 0.05 Mbps.

7 Summary

The two Actor-Critic variants specified here share the same MDP interface but differ in memory footprint ($\approx$7.1 MB tabular vs $\approx$40 kB neural parameters), per-TTI compute ($O(1)$ vs $O(D)$), and stability requirements. The tabular version is preferred when the state space is discrete and small; the deep version is preferred when the state grows or continuous interpolation is needed. Both benefit from parameter sharing across users when the problem has multi-user structure.

The eight non-obvious tricks in Section 4 are what separate a working implementation from a divergent one. Each was diagnosed empirically during the v1–v6 iterations of this project and is documented in the project audit for reproducibility.

References

[1] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in *Proc. ICML*, 2016.

[2] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[3] J. Schulman et al., "High-dimensional continuous control using generalized advantage estimation," in *Proc. ICLR*, 2016.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[5] R. Bruno, A. Masaracchia, and A. Passarella, "Robust adaptive modulation and coding (AMC) selection in LTE systems using reinforcement learning," in *Proc. IEEE VTC-Fall*, 2014.